# Practical No 1

## Aim: Breadth First Search & Iterative Depth First Search ..

**Dataset: RMP.py File**

**Program 1 :Implement the Breadth First Search algorithm to solve a given problem.**

Dataset: -RMP.py File

Requirements: - RMP.py, Idle Python

```
import queue as Q
from RMP import dict_gn
start = 'Arad'
goal = "Bucharest"
result=''
def BFS(city,cityq,visitedq):
    global result
    if city==start:
        result = result + "" + city
        for eachcity in dict_gn[city].keys():
            if eachcity==goal:
                result = result + " " + eachcity
                return
            if eachcity not in cityq.queue and eachcity not in visitedq.queue:
                cityq.put(eachcity)
                result = result + " " + eachcity
                visitedq.put(city)
                BFS(cityq.get(),cityq,visitedq)
def main():
    cityq = Q.Queue()
    visitedq = Q.Queue()
    BFS(start,cityq,visitedq)
    print("BFS Traversal From ", start," to " , goal, "is :")
    print(result)
main()
```

**Output:**

```
BFS Traversal From  Arad  to  Bucharest is :
Arad Zerind Timisoara Sibiu
```

**Program 2 : Implement the Iterative Depth First Search algorithm to solve the same problem.**

```
import queue as Q
from RMP import dict_gn
start = "Arad"
goal = "Bucharest"
result = ""
def DLS(city,visitedstack,startlimit,endlimit):
    global result
    found = 0
    result = result + city + " "
    visitedstack.append(city)
    if city == goal:
        return 1
    if startlimit == endlimit:
        return 0
    for eachcity in dict_gn[city].keys():
        if eachcity not in visitedstack:
            found = DLS(eachcity,visitedstack,startlimit+1,endlimit)
            if found:
                return found
def IDDFS(city,visitedstack,endlimit):
    global result
    for i in range(0,endlimit):
        print("Seaching at Limit:", i)
        found = DLS(city,visitedstack, 0 , i)
        if found:
            print("Found")
            break
        else:
            print("Not Found!")
            print(result)
            print("______")
            result=""
            visitedstack = []

def main():
    visitedstack = []
    IDDFS(start,visitedstack,9)
    print("IDDFS Traversal from ", start, " to ",goal," is:")
    print(result)
main()
```

## Output:

```
Seaching at Limit: 0
Not Found!
Arad

______
Seaching at Limit: 1
Not Found!
Arad Zerind Timisoara Sibiu

______
Seaching at Limit: 2
Not Found!
Arad Zerind Oradea Timisoara Lugoj Sibiu Rimnicu Fagaras

______
Seaching at Limit: 3
Not Found!
Arad Zerind Oradea Sibiu Timisoara Lugoj Mehadia

______
Seaching at Limit: 4
Not Found!
Arad Zerind Oradea Sibiu Rimnicu Fagaras Timisoara Lugoj Mehadia Drob

______
Seaching at Limit: 5
Found
IDDFS Traversal from  Arad  to  Bucharest  is:
Arad Zerind Oradea Sibiu Rimnicu Pitesti Craiova Fagaras Bucharest
```

# Practical No. 2

## Aim: A* Search and Recursive Best-First Search :-

**Dataset: RMP.py File**

**Program 1 :Implement the A* Search algorithm for solving a pathfinding problem.**

```
import queue as Q
from RMP import dict_gn
from RMP import dict_hn

start = 'Arad'
goal = 'Bucharest'
result = ''
def get_fn(citystr):
    cities = citystr.split(",")
    hn = gn = 0
    for ctr in range(0, len(cities) - 1):
        gn = gn + dict_gn[cities[ctr]][cities[ctr + 1]]
    hn = dict_hn[cities[len(cities) - 1]]
    return (hn + gn)
def expand(cityq):
    global result
    tot, citystr, thiscity = cityq.get()
    if thiscity == goal:
        result = citystr + "::" + str(tot)
        return
    for cty in dict_gn[thiscity]:
        cityq.put((get_fn(citystr + "," + cty), citystr + "," + cty, cty))
    expand(cityq)
def main():
    cityq = Q.PriorityQueue()
    thiscity = start
    cityq.put((get_fn(start), start, thiscity))
    expand(cityq)
    print("The A* path with the total is: ")
    print(result)
main()
```

## Output:

```
======== RESTART: E:/TYCS PRACTICAL SEM5/AIpr
The A* path with the total is:
Arad,Sibiu,Rimnicu,Pitesti,Bucharest::418
>>>
```

**Program 2: Implement the Recursive Best-First Search algorithm for the same problem.**

```
import queue as Q
from RMP import dict_gn
from RMP import dict_hn
start = 'Arad'
goal = 'Bucharest'
result = ''
def get_fn(citystr):
    cities = citystr.split(",")
    hn = gn = 0
    for ctr in range(0, len(cities) - 1):
        gn = gn + dict_gn[cities[ctr]][cities[ctr + 1]]
    hn = dict_hn[cities[len(cities) - 1]]
    return (hn + gn)
def printout(cityq):
    for i in range(0, cityq.qsize()):
        print(cityq.queue[i])
def expand(cityq):
    global result
    tot, citystr, thiscity = cityq.get()
    nexttot = 999
    if not cityq.empty():
        nexttot, nextcitystr, nextthiscity = cityq.queue[0]
    if thiscity == goal and tot < nexttot:
        result = citystr + "::" + str(tot)
        return
    print("Expaded city ---------", thiscity)
    print("second best f(n)---------", nexttot)
    tempq = Q.PriorityQueue()
    for cty in dict_gn[thiscity]:
        tempq.put((get_fn(citystr + ',' + cty), citystr + ',' + cty, cty))
    for ctr in range(1, 3):
        ctrtot, ctrcitystr, ctrthiscity = tempq.get()
        if ctrtot < nexttot:
            cityq.put((ctrtot, ctrcitystr, ctrthiscity))
        else:
            cityq.put((ctrtot, citystr, thiscity))
```

```
        break
    printout(cityq)
    expand(cityq)
def main():
    cityq = Q.PriorityQueue()
    thiscity = start
    cityq.put((999, "NA", "NA"))
    cityq.put((get_fn(start), start, thiscity))
    expand(cityq)
print(result)
```

## Output:

```
======== RESTART: E:/TYCS PRACTICAL SEM5/AIpractical/1mpaip
Expaded city --------- Arad
second best f(n)--------- 999
(393, 'Arad,Sibiu', 'Sibiu')
(999, 'NA', 'NA')
Expaded city --------- Sibiu
second best f(n)--------- 999
(413, 'Arad,Sibiu,Rimnicu', 'Rimnicu')
(999, 'NA', 'NA')
Expaded city --------- Rimnicu
second best f(n)--------- 999
(417, 'Arad,Sibiu,Rimnicu,Pitesti', 'Pitesti')
(999, 'NA', 'NA')
Expaded city --------- Pitesti
second best f(n)--------- 999
(418, 'Arad,Sibiu,Rimnicu,Pitesti,Bucharest', 'Bucharest')
(999, 'NA', 'NA')
Arad,Sibiu,Rimnicu,Pitesti,Bucharest::418
>>>
```

# Practical No 3

## Aim: Decision Tree Learning :

- **Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.**
- **Evaluate the accuracy and effectiveness of the decision tree on test data.**
- **Visualize and interpret the generated decision tree.**

## Program 1: -

```
import numpy as np
import pandas as pd
PlayTennis = pd.read_csv("D:\AI TYCS AA\pract3\PlayTennis.csv")
print(PlayTennis)
```

## Output:

```
In [4]: import numpy as np
        import pandas as pd

In [7]: PlayTennis = pd.read_csv("D:\AI TYCS AA\pract3\PlayTennis.csv")
        print(PlayTennis)
```

| | Outlook | Temperature | Humidity | Wind | Play Tennis |
|---|---|---|---|---|---|
| 0 | Sunny | Hot | High | Weak | No |
| 1 | Sunny | Hot | High | Strong | No |
| 2 | Overcast | Hot | High | Weak | Yes |
| 3 | Rain | Mild | High | Weak | Yes |
| 4 | Rain | Cool | Normal | Weak | Yes |
| 5 | Rain | Cool | Normal | Strong | No |
| 6 | Overcast | Cool | Normal | Strong | Yes |
| 7 | Sunny | Mild | High | Weak | No |
| 8 | Sunny | Cool | Normal | Weak | Yes |
| 9 | Rain | Mild | Normal | Weak | Yes |
| 10 | Sunny | Mild | Normal | Strong | Yes |
| 11 | Overcast | Mild | High | Strong | Yes |
| 12 | Overcast | Hot | Normal | Weak | Yes |
| 13 | Rain | Mild | High | Strong | No |

## Program 2:-

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
PlayTennis['Outlook'] = le.fit_transform(PlayTennis['Outlook'])
PlayTennis['Temperature'] = le.fit_transform(PlayTennis['Temperature'])
PlayTennis['Humidity'] = le.fit_transform(PlayTennis['Humidity'])
PlayTennis['Wind'] = le.fit_transform(PlayTennis['Wind'])
```

PlayTennis['Play Tennis'] = le.fit_transform(PlayTennis['Play Tennis'])
Print(PlayTennis)

## Output:

```
In [12]: from sklearn.preprocessing import LabelEncoder
         le = LabelEncoder()
         PlayTennis['Outlook'] = le.fit_transform(PlayTennis['Outlook'])
         PlayTennis['Temperature'] = le.fit_transform(PlayTennis['Temperature'])
         PlayTennis['Humidity'] = le.fit_transform(PlayTennis['Humidity'])
         PlayTennis['Wind'] = le.fit_transform(PlayTennis['Wind'])
         PlayTennis['Play Tennis'] = le.fit_transform(PlayTennis['Play Tennis'])
         PlayTennis
```

Out[12]:

| | Outlook | Temperature | Humidity | Wind | Play Tennis |
|---|---|---|---|---|---|
| 0 | 2 | 1 | 0 | 1 | 0 |
| 1 | 2 | 1 | 0 | 0 | 0 |
| 2 | 0 | 1 | 0 | 1 | 1 |
| 3 | 1 | 2 | 0 | 1 | 1 |
| 4 | 1 | 0 | 1 | 1 | 1 |
| 5 | 1 | 0 | 1 | 0 | 0 |
| 6 | 0 | 0 | 1 | 0 | 1 |
| 7 | 2 | 2 | 0 | 1 | 0 |
| 8 | 2 | 0 | 1 | 1 | 1 |
| 9 | 1 | 2 | 1 | 1 | 1 |
| 10 | 2 | 2 | 1 | 0 | 1 |
| 11 | 0 | 2 | 0 | 0 | 1 |
| 12 | 0 | 1 | 1 | 1 | 1 |
| 13 | 1 | 2 | 0 | 0 | 0 |

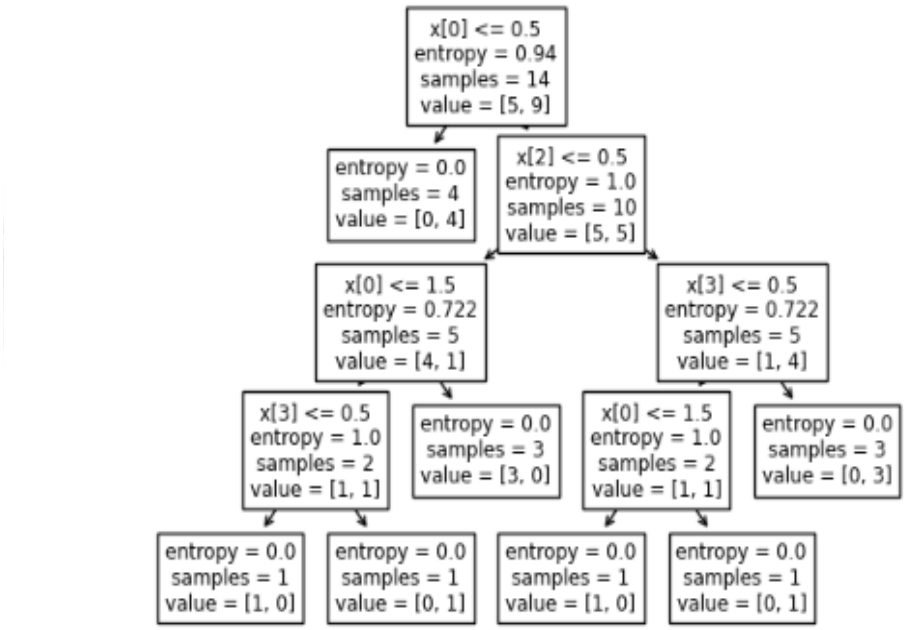
## Program 3:-

y = PlayTennis['Play Tennis']
x = PlayTennis.drop(['Play Tennis'],axis=1)
from sklearn import tree
clf = tree.DecisionTreeClassifier(criterion = 'entropy')
clf = clf.fit(x,y)
tree.plot_tree(clf)

```
In [16]: y = PlayTennis['Play Tennis']
         x = PlayTennis.drop(['Play Tennis'],axis=1)
         from sklearn import tree
         clf = tree.DecisionTreeClassifier(criterion = 'entropy')
         clf = clf.fit(x,y)
         tree.plot_tree(clf)
```

## Output:-

```
Out[16]: [Text(0.4444444444444444, 0.9, 'x[0] <= 0.5\nentropy = 0.94\nsamples = 14\nvalue = [5, 9]'),
          Text(0.3333333333333333, 0.7, 'entropy = 0.0\nsamples = 4\nvalue = [0, 4]'),
          Text(0.5555555555555556, 0.7, 'x[2] <= 0.5\nentropy = 1.0\nsamples = 10\nvalue = [5, 5]'),
          Text(0.3333333333333333, 0.5, 'x[0] <= 1.5\nentropy = 0.722\nsamples = 5\nvalue = [4, 1]'),
          Text(0.2222222222222222, 0.3, 'x[3] <= 0.5\nentropy = 1.0\nsamples = 2\nvalue = [1, 1]'),
          Text(0.1111111111111111, 0.1, 'entropy = 0.0\nsamples = 1\nvalue = [1, 0]'),
          Text(0.3333333333333333, 0.1, 'entropy = 0.0\nsamples = 1\nvalue = [0, 1]'),
          Text(0.4444444444444444, 0.3, 'entropy = 0.0\nsamples = 3\nvalue = [3, 0]'),
          Text(0.7777777777777778, 0.5, 'x[3] <= 0.5\nentropy = 0.722\nsamples = 5\nvalue = [1, 4]'),
          Text(0.6666666666666666, 0.3, 'x[0] <= 1.5\nentropy = 1.0\nsamples = 2\nvalue = [1, 1]'),
          Text(0.5555555555555556, 0.1, 'entropy = 0.0\nsamples = 1\nvalue = [1, 0]'),
          Text(0.7777777777777778, 0.1, 'entropy = 0.0\nsamples = 1\nvalue = [0, 1]'),
          Text(0.8888888888888888, 0.3, 'entropy = 0.0\nsamples = 3\nvalue = [0, 3]')]
```




## Program 4:-

```
y = PlayTennis['Play Tennis']
x = PlayTennis.drop(['Play Tennis'],axis=1)
print(x)
from sklearn import tree
clf = tree.DecisionTreeClassifier(criterion = 'entropy')
clf = clf.fit(x,y)
print(clf)
tree.plot_tree(clf)
```

## Output:-

In [17]:
```
y = PlayTennis['Play Tennis']
x = PlayTennis.drop(['Play Tennis'],axis=1)
print(x)
```

| | Outlook | Temperature | Humidity | Wind |
|---|---|---|---|---|
| 0 | 2 | 1 | 0 | 1 |
| 1 | 2 | 1 | 0 | 0 |
| 2 | 0 | 1 | 0 | 1 |
| 3 | 1 | 2 | 0 | 1 |
| 4 | 1 | 0 | 1 | 1 |
| 5 | 1 | 0 | 1 | 0 |
| 6 | 0 | 0 | 1 | 0 |
| 7 | 2 | 2 | 0 | 1 |
| 8 | 2 | 0 | 1 | 1 |
| 9 | 1 | 2 | 1 | 1 |
| 10 | 2 | 2 | 1 | 0 |
| 11 | 0 | 2 | 0 | 0 |
| 12 | 0 | 1 | 1 | 1 |
| 13 | 1 | 2 | 0 | 0 |

In [18]:
```
from sklearn import tree
clf = tree.DecisionTreeClassifier(criterion = 'entropy')
clf = clf.fit(x,y)
print(clf)
```

```
DecisionTreeClassifier(criterion='entropy')
```

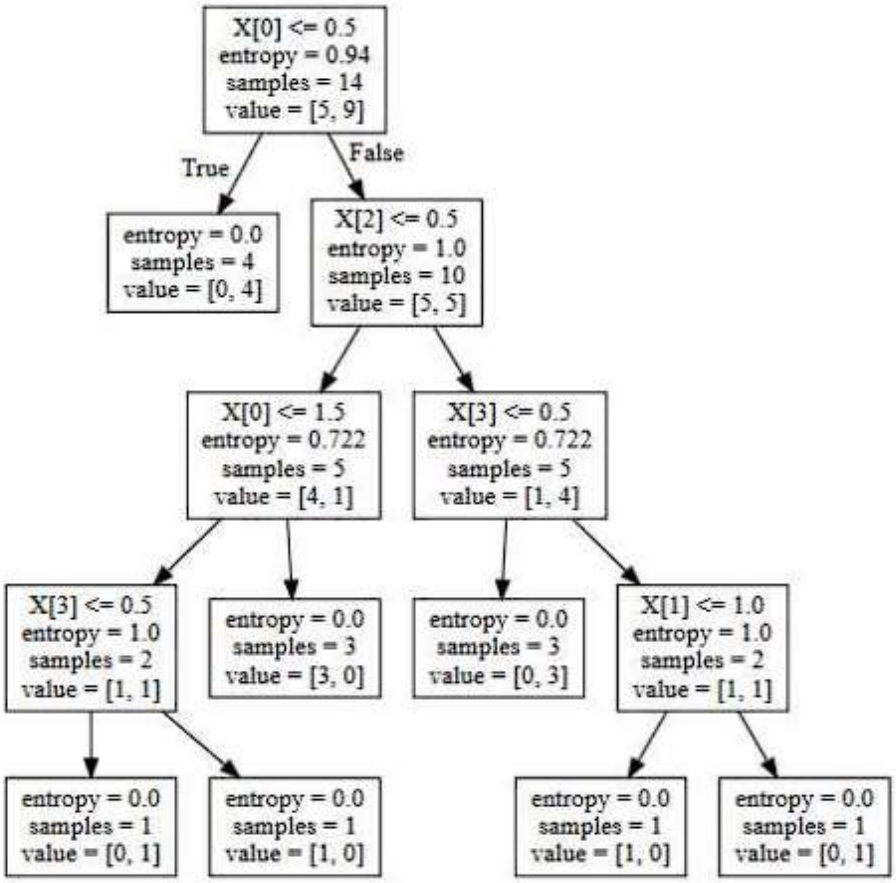
## Program 5:-

- Lets split the training data and its coresponding prediction values.
- y - holds all the decisions.
- X - holds the training data.

```
y= PlayTennis['play']
X PlayTennis.drop(['play'], axis=1)
#Fitting the model
from sklearn import tree
clf  = tree.DecisionTreeClassifier(criterion = 'entropy')
clf = clf.fit(X, y)
# We can visualize the tree using tree.plot_ Tree
tree.plot_tree(clf)
import graphviz
dot_data tree.export_graphviz(clf, out_file=None)
graph = graphviz.Source(dot_data)
graph
```

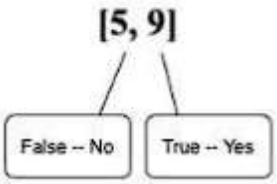
# Output:-

Out[9]:




In the above graph,

- X[0] -> Outlook
- X[1] -> Temperature
- X[2] -> Humidity
- X[3] -> Wind




values

Since we dont have any data to test. we can just make the model to predict our train data.

# Program 6:-

```
X_pred = clf.predict(X)
X_pred == y
```

## Output:-

```
0     True
1     True
2     True
3     True
4     True
5     True
6     True
7     True
8     True
9     True
10    True
11    True
12    True
13    True
Name: play, dtype: bool
```

# Practical No 4

## Aim: Feed Forward Back Propagation Neural Network

- **Implement the Feed Forward Back Propagation algorithm to train a neural network.**
- **Use a given dataset to train the neural network for a specific task.**

## Code: - Implement the Feed Forward Back Propagation algorithm to train a neural network.

```
from doctest import OutputChecker
import numpy as np
class NeuralNetwork():
    def __init__(self):
        np.random.seed()
        self.synaptic_weights=2*np.random.random((3,1))-1
    def sigmoid(self,x):
        return 1/(1+np.exp(-x))
    def sigmoid_derivative(self,x):
        return x*(1-x)
    def train(self,training_inputs,training_outputs,training_iterations):
        for iteration in range(training_iterations):
            output=self.think(training_inputs)
            error = training_outputs-output
            adjustments=np.dot(training_inputs.T,error*self.sigmoid_derivative(output))
            self.synaptic_weights +=adjustments
    def think(self,inputs):
        inputs=inputs.astype(float)
        output=self.sigmoid(np.dot(inputs,self.synaptic_weights))
        return output
if __name__ == "__main__":
    #initializing the neuron class
    neural_network = NeuralNetwork()
    print("Beginning Randomly Generated Weights: ")
    print(neural_network.synaptic_weights)
#training data consisting of 4 examples--3 input values and 1 output
training_inputs = np.array([[0,0,1],
[1,1,1],
[1,0,1],
[0,1,1]])
training_outputs = np.array([[0,1,1,0]]).T
#training taking place
neural_network.train(training_inputs, training_outputs, 15000)
print("Ending Weights After Training: ")
```

```
print(neural_network.synaptic_weights)
user_input_one = str(input("User Input One: "))
user_input_two = str(input("User Input Two: "))
user_input_three = str(input("User Input Three: "))
print("Considering New Situation: ", user_input_one, user_input_two, user_input_three)
print("New Output data: ")
print(neural_network.think(np.array([user_input_one, user_input_two, user_input_three])))
```

## Output:

```
Beginning Randomly Generated Weights:
[[-0.12975441]
 [ 0.41752384]
 [-0.91671975]]
Ending Weights After Training:
[[10.08740549]
 [-0.20698344]
 [-4.83755885]]
User Input One: 12
User Input Two: 23
User Input Three: 16
Considering New Situation:  12 23 16
New Output data:
[1.]
```

# Practical No. 5

## Aim: Support Vector Machines (SVM)

- **Implement the SVM algorithm for binary classification.**
- **Train an SVM model using a given dataset and optimize its parameters.**
- **Evaluate the performance of the SVM model on test data and analyse the results.**

### Program 1: Importing Libraries:

Pip install skompiler
from warnings import filterwarnings
filterwarnings("Ignore")

### Output:

```
Collecting skompiler
  Downloading SKompiler-0.6.tar.gz (45 kB)
     |████████████████████████████████| 45 kB 388 kB/s
Requirement already satisfied: scikit-learn>=0.22 in /opt/conda/lib/python3.7/sit
e-packages (from skompiler) (0.23.2)
Requirement already satisfied: joblib>=0.11 in /opt/conda/lib/python3.7/site-pack
ages (from scikit-learn>=0.22->skompiler) (1.0.1)
Requirement already satisfied: scipy>=0.19.1 in /opt/conda/lib/python3.7/site-pac
kages (from scikit-learn>=0.22->skompiler) (1.6.3)
Requirement already satisfied: numpy>=1.13.3 in /opt/conda/lib/python3.7/site-pac
kages (from scikit-learn>=0.22->skompiler) (1.19.5)
Requirement already satisfied: threadpoolctl>=2.0.0 in /opt/conda/lib/python3.7/s
ite-packages (from scikit-learn>=0.22->skompiler) (2.1.0)
Building wheels for collected packages: skompiler
  Building wheel for skompiler (setup.py) ... -⊟ ⊟\⊟ ⊟done
  Created wheel for skompiler: filename=SKompiler-0.6-py3-none-any.whl size=54265
sha256=940fba6a64063cef9232f47a844962bf8f3f142124b4629cea6a480f4af9acbc
  Stored in directory: /root/.cache/pip/wheels/47/1c/59/b80a730f4afd2144bad854df4
b167b812486c9d4c1bd4cf4c5
Successfully built skompiler
Installing collected packages: skompiler
Successfully installed skompiler-0.6
WARNING: Running pip as root will break packages and permissions. You should inst
all packages reliably by using venv: https://pip.pypa.io/warnings/venv
Note: you may need to restart the kernel to use updated packages.
```

### Program 2:Support Vector Machine- Classifier (SVM)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf  # Corrected from 'simf' to 'smf'

```
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split, cross_val_score  # Removed
'cross_val_' as it is incomplete
from sklearn.decomposition import PCA
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import scale
from sklearn.metrics import roc_auc_score, roc_curve, classification_report,
confusion_matrix, accuracy_score  # Combined imports for clarity
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier  #
Corrected 'GradientBoostin' to 'GradientBoostingClassifier'
from sklearn.svm import SVC, LinearSVC
import time
from matplotlib.colors import ListedColormap
from xgboost import XGBRegressor
from skompiler import skompile
from lightgbm import LGBMRegressor

pd.set_option('display.max_rows', 1000)
pd.set_option('display.max_columns", 1000)
pd.set_option('display.width', 1000)
```

## Output:

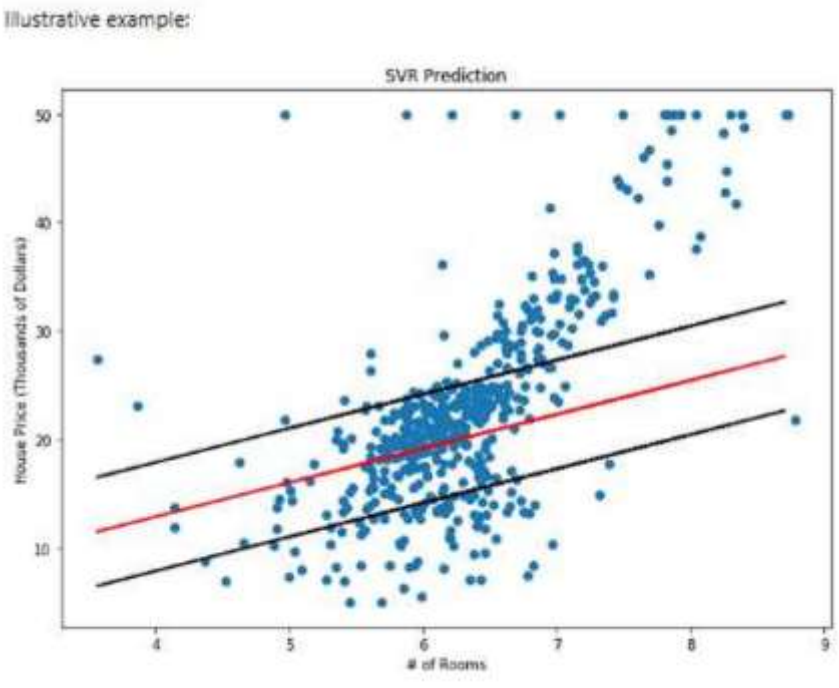


Photo is cited by:https://towardsdatascience.com/an-introduction-to-support-vector-regression-svr-a3ebc1672c2

```
df = pd.read_csv("../input/pima-indians-diabetes-database/diabetes.csv")
df.head()
```

## Program:

Photo is cited by:https://towardsdatascience.com/an-introduction-to-support-vector-regression-svr-a3ebc1672c2

df= pd.read_csv("../input/pima-indians-diabetes-database/diabetes.csv")

df.head()

| | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI | DiabetesPedigreeF |
|---|---|---|---|---|---|---|---|
| 0 | 6 | 148 | 72 | 35 | 0 | 33.6 | |
| 1 | 1 | 85 | 66 | 29 | 0 | 26.6 | |
| 2 | 8 | 183 | 64 | 0 | 0 | 23.3 | |
| 3 | 1 | 89 | 66 | 23 | 94 | 28.1 | |
| 4 | 0 | 137 | 40 | 35 | 168 | 43.1 | |

df.shape

(768, 9)

df.describe()

```
In [6]: df.shape
Out[6]: (768, 9)
In [7]: df.describe()
```

Out[7]:

| | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI |
|---|---|---|---|---|---|---|
| count | 768.000000 | 768.000000 | 768.000000 | 768.000000 | 768.000000 | 768.000000 |
| mean | 3.845052 | 120.894531 | 69.105469 | 20.536458 | 79.799479 | 31.992578 |
| std | 3.369578 | 31.972618 | 19.355807 | 15.952218 | 115.244002 | 7.884160 |
| min | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 |
| 25% | 1.000000 | 99.000000 | 62.000000 | 0.000000 | 0.000000 | 27.300000 |
| 50% | 3.000000 | 117.000000 | 72.000000 | 23.000000 | 30.500000 | 32.000000 |
| 75% | 6.000000 | 140.250000 | 80.000000 | 32.000000 | 127.250000 | 36.600000 |
| max | 17.000000 | 199.000000 | 122.000000 | 99.000000 | 846.000000 | 67.100000 |

**Program:**

```
# Dropping the 'Outcome' column and separating features and target variable
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Splitting the data into training and testing sets
X_train = X.iloc[:600]  # Corrected 'x' to 'X'
X_test = X.iloc[600:]   # Added '=' to assign the value to X_test
y_train = y[:600]       # Added '=' to assign the value to y_train
y_test = y[600:]        # Added '=' to assign the value to y_test

# Printing the shapes of the training and testing sets
print("X_train Shape: ", X_train.shape)
print("X_test Shape: ", X_test.shape)
print("y_train Shape: ", y_train.shape)
print("y_test Shape: ", y_test.shape)
```

**Output:**

```
X_train Shape:  (600, 8)
X_test Shape:  (168, 8)
y_train Shape:  (600,)
y_test Shape:  (168,)
```

**Program:**

```
support_vector_classifier = SVC(kernel="linear")
support_vector_classifier.fit(X_train, y_train)
support_vector_classifier
```

```
SVC(kernel='linear')
```

```
support_vector_classifier.C
```

```
1.0
```

**Program 3: Prediction**

```
y_pred = support_vector_classifier.predict(X_test)
cm = confusion_matrix(y_test,y_pred)
cm
```

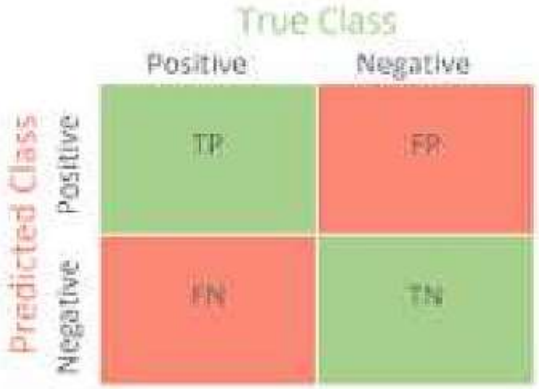
```
array([[96, 12],
       [27, 33]])
```

print("Our Accuracy is: ", (cm[0][0] + cm[1][1]) / (cm[0][0] + cm[1][1] + cm[0][1] + cm[1][0]))

accuracy_score(y_test,y_pred)

**Output:**

- **true positive:** for correctly predicted event values.
- **false positive:** for incorrectly predicted event values.
- **true negative:** for correctly predicted no-event values.
- **false negative:** for incorrectly predicted no-event values.




print("Our Accuracy is: ", (cm[0][0] + cm[1][1]) / (cm[0][0] + cm[1][1] + cm[0][1] + cm[1][0]))

```
Our Accuracy is:  0.7678571428571429
```

accuracy_score(y_test,y_pred)

```
0.7678571428571429
```

print(classification_report(y_test,y_pred))

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.78 | 0.89 | 0.83 | 108 |
| 1 | 0.73 | 0.55 | 0.63 | 60 |
| accuracy | | | 0.77 | 168 |
| macro avg | 0.76 | 0.72 | 0.73 | 168 |
| weighted avg | 0.76 | 0.77 | 0.76 | 168 |

## Model Tuning & Validation

support_vector_classifier

```
SVC(kernel='linear')
```

```
accuracies cross_val_score (estimator support_vector_classifier,
X=X_train,
y=y_train, cv=10)
print("Average Accuracy: (:.2f} %".format(accuracies.mean()*100))
print("Standart Deviation of Accuracies: {:.2f} %".format(accuracies.std()*100))
```

```
Average Accuracy: 77.33 %
Standart Deviation of Accuracies: 4.90 %
```

```
support_vector_classifier.predict(X_test)
svm_params ={"C":np.arange(1,20)}

support_vector_classifier.predict(X_test)[:10]
```

```
array([0, 0, 0, 1, 1, 0, 1, 0, 1, 0])
```

```
svm_params ={"C":np.arange(1,20)}
svm = SVC(kernel="linear")
from sklearn.model_selection import GridSearchCV

# Assuming 'svm' is your SVC model and 'svm_params' is a dictionary of parameters
svm_cv = GridSearchCV(estimator=svm, param_grid=svm_params, cv=8)

import time  # Ensure you have imported the time module

start_time = time.time()
svm_cv.fit(X_train, y_train)
elapsed_time = time.time() - start_time
print(f"Elapsed time for Support Vector Regression cross-validation: {elapsed_time:.3f} seconds")
```

svm_cv.best_score_

```
0.7716666666666667
```

svm_cv.best_params_

```
{'C': 2}
```

svm_tuned SVC(kernel="linear", C=2).fit(X_train,y_train)
svm_tuned

```
SVC(C=2, kernel='linear')
```

SVC(C=2, kernel='linear')
y_pred svm_tuned.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
cm

```
array([[96, 12],
       [27, 33]])
```

- **true positive:** for correctly predicted event values.
- **false positive:** for incorrectly predicted event values.
- **true negative:** for correctly predicted no-event values.
- **false negative:** for incorrectly predicted no-event values.

print("Our Accuracy is: ", (cm[0][0] + cm[1][1]) / (cm[0][0] + cm[1][1] + cm[0][1] + cm[1][0]))

accuracy_score(y_test,y_pred)
print(classification_report(y_test,y_pred))

```
              precision    recall  f1-score   support

           0       0.78      0.89      0.83       108
           1       0.73      0.55      0.63        60

    accuracy                           0.77       168
   macro avg       0.76      0.72      0.73       168
weighted avg       0.76      0.77      0.76       168
```

# Practical No . 6

## Aim: Ad boost Ensemble Learning

- **Implement the Adaboost algorithm to create an ensemble of weak classifiers.**
- **Train the ensemble model on a given dataset and evaluate its performance.**
- **Compare the results with individual weak classifiers.**

## Program:

```
import pandas as pd
from sklearn import model_selection
from sklearn.ensemble import AdaBoostClassifier

# Correct URL for the dataset
url="https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians
diabetes.data.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
dataframe = pd.read_csv(url, names=names)
array = dataframe.values
X = array[:, 0:8]
Y = array[:, 8]
seed = 7
num_trees = 30

# kfold makes trees with split number.
kfold = model_selection.KFold(n_splits=10, random_state=seed, shuffle=True)

# Set the algorithm to 'SAMME' to avoid deprecation warnings
model = AdaBoostClassifier(n_estimators=num_trees, random_state=seed,
algorithm='SAMME')

# cross_val_score method is used to calculate the accuracy of the model sliced into X, Y
results = model_selection.cross_val_score(model, X, Y, cv=kfold)
print("Mean Accuracy: ", results.mean())
```

## Output:

```
========== RESTART: H:/T
Mean Accuracy:  0.7552460697197538
```

# Practical No. 7

## Aim: Naive Bayes' Classifier

- **Implement the Naive Bayes' algorithm for classification.**
- **Train a Naive Bayes' model using a given dataset and calculate class probabilities.**
- **Evaluate the accuracy of the model on test data and analyze the results.**

**Requirement: disease dataset**

## Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, CategoricalNB, GaussianNB
from sklearn.metrics import accuracy_score

df = pd.read_csv('H:\TYCS PRACTICAL SEM5\AIpractical\diabetes.csv')

df.head(11)
```

## Output:

| | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | target |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 63 | 1 | 3 | 145 | 233 | 1 | 0 | 150 | 0 | 2.3 | 0 | 0 | 1 | 1 |
| 1 | 37 | 1 | 2 | 130 | 250 | 0 | 1 | 187 | 0 | 3.5 | 0 | 0 | 2 | 1 |
| 2 | 41 | 0 | 1 | 130 | 204 | 0 | 0 | 172 | 0 | 1.4 | 2 | 0 | 2 | 1 |
| 3 | 56 | 1 | 1 | 120 | 236 | 0 | 1 | 178 | 0 | 0.8 | 2 | 0 | 2 | 1 |
| 4 | 57 | 0 | 0 | 120 | 354 | 0 | 1 | 163 | 1 | 0.6 | 2 | 0 | 2 | 1 |
| 5 | 57 | 1 | 0 | 140 | 192 | 0 | 1 | 148 | 0 | 0.4 | 1 | 0 | 1 | 1 |
| 6 | 56 | 0 | 1 | 140 | 294 | 0 | 0 | 153 | 0 | 1.3 | 1 | 0 | 2 | 1 |
| 7 | 44 | 1 | 1 | 120 | 263 | 0 | 1 | 173 | 0 | 0.0 | 2 | 0 | 3 | 1 |
| 8 | 52 | 1 | 2 | 172 | 199 | 1 | 1 | 162 | 0 | 0.5 | 2 | 0 | 3 | 1 |
| 9 | 57 | 1 | 2 | 150 | 168 | 0 | 1 | 174 | 0 | 1.6 | 2 | 0 | 2 | 1 |
| 10 | 54 | 1 | 0 | 140 | 239 | 0 | 1 | 160 | 0 | 1.2 | 2 | 0 | 2 | 1 |

## Code:

```
df.tail()
```

## Output:

| | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | target |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 298 | 57 | 0 | 0 | 140 | 241 | 0 | 1 | 123 | 1 | 0.2 | 1 | 0 | 3 | 0 |
| 299 | 45 | 1 | 3 | 110 | 264 | 0 | 1 | 132 | 0 | 1.2 | 1 | 0 | 3 | 0 |
| 300 | 68 | 1 | 0 | 144 | 193 | 1 | 1 | 141 | 0 | 3.4 | 1 | 2 | 3 | 0 |
| 301 | 57 | 1 | 0 | 130 | 131 | 0 | 1 | 115 | 1 | 1.2 | 1 | 1 | 3 | 0 |
| 302 | 57 | 0 | 1 | 130 | 236 | 0 | 0 | 174 | 0 | 0.0 | 1 | 1 | 2 | 0 |

## Code:

```
df.info()
```

## Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       303 non-null    int64  
 1   sex       303 non-null    int64  
 2   cp        303 non-null    int64  
 3   trestbps  303 non-null    int64  
 4   chol      303 non-null    int64  
 5   fbs       303 non-null    int64  
 6   restecg   303 non-null    int64  
 7   thalach   303 non-null    int64  
 8   exang     303 non-null    int64  
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64  
 11  ca        303 non-null    int64  
 12  thal      303 non-null    int64  
 13  target    303 non-null    int64  
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

## Code:

```
from sklearn.preprocessing import LabelEncoder
# Initialize the LabelEncoder
le = LabelEncoder()
# Print the DataFrame shape and column names
print("DataFrame shape:", df.shape)
print("Columns in DataFrame:", df.columns.tolist())
# Strip whitespace from column names
df.columns = df.columns.str.strip()
# Check if the columns exist before dropping NaN values
```

```
missing_columns = [col for col in ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
'restecg','thalach','exang','oldpeake','slop','ca','thal','target'] if col not in df.columns]
if missing_columns:
    print("Missing columns:", missing_columns)
else:
    # Handle missing values (if any)
    df = df.dropna(subset=['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
'restecg','thalach','exang','oldpeake','slop','ca','thal','target'])
    # Transform categorical columns using LabelEncoder
    df['age'] = le.fit_transform(df['age'])
    df['sex'] = le.fit_transform(df['sex'])
    df['cp'] = le.fit_transform(df['cp'])
    df['trestbps'] = le.fit_transform(df['trestbps'])
    df['chol'] = le.fit_transform(df['chol'])
    df['fbs'] = le.fit_transform(df['fbs'])
    df['restecg'] = le.fit_transform(df['restecg'])
    df['thalach'] = le.fit_transform(df['thalach'])
    df['exang'] = le.fit_transform(df['exang'])
    df['oldpeake'] = le.fit_transform(df['oldpeake'])
    df['slop'] = le.fit_transform(df['slop'])
    df['ca'] = le.fit_transform(df['ca'])
    df['thal'] = le.fit_transform(df['thal'])
    df['target'] = le.fit_transform(df['target'])
```

## Output:

```
DataFrame shape: (303, 14)
Columns in DataFrame: ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca',
'thal', 'target']
Missing columns: ['oldpeake', 'slop']
```

---

## Code:

```
df.info()
```

## Ouput:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

## Code:

df.head(11)

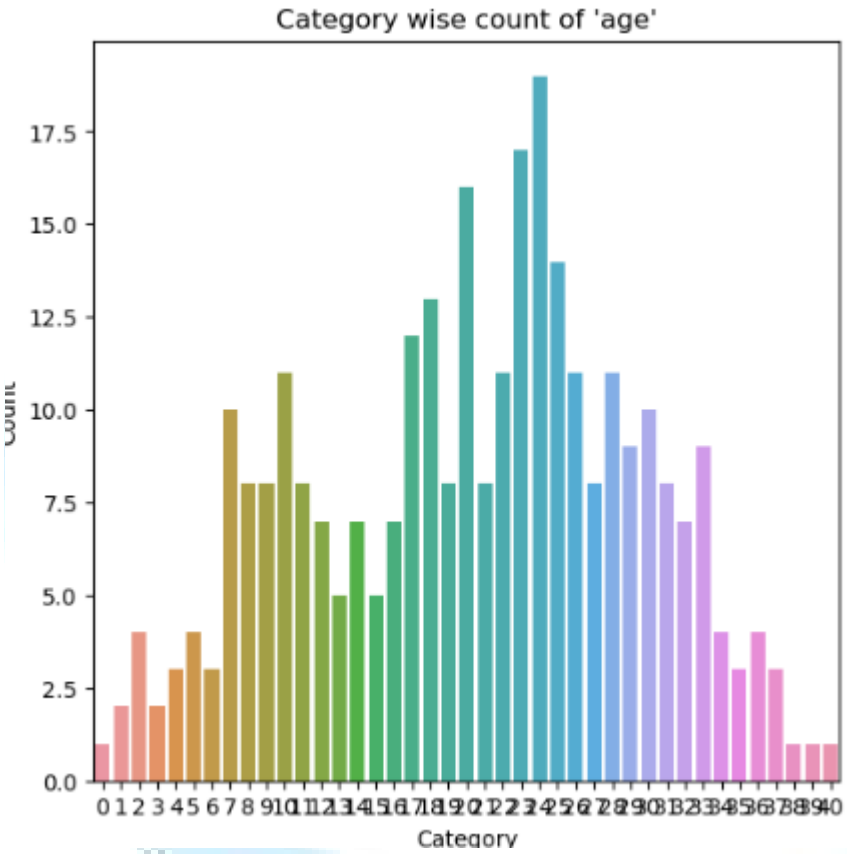
## Output:

| | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | target |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 29 | 1 | 3 | 31 | 64 | 1 | 0 | 150 | 0 | 2.3 | 0 | 0 | 1 | 1 |
| 1 | 3 | 1 | 2 | 22 | 80 | 0 | 1 | 187 | 0 | 3.5 | 0 | 0 | 2 | 1 |
| 2 | 7 | 0 | 1 | 22 | 35 | 0 | 0 | 172 | 0 | 1.4 | 2 | 0 | 2 | 1 |
| 3 | 22 | 1 | 1 | 14 | 67 | 0 | 1 | 178 | 0 | 0.8 | 2 | 0 | 2 | 1 |
| 4 | 23 | 0 | 0 | 14 | 145 | 0 | 1 | 163 | 1 | 0.6 | 2 | 0 | 2 | 1 |
| 5 | 23 | 1 | 0 | 28 | 25 | 0 | 1 | 148 | 0 | 0.4 | 1 | 0 | 1 | 1 |
| 6 | 22 | 0 | 1 | 28 | 116 | 0 | 0 | 153 | 0 | 1.3 | 1 | 0 | 2 | 1 |
| 7 | 10 | 1 | 1 | 14 | 92 | 0 | 1 | 173 | 0 | 0.0 | 2 | 0 | 3 | 1 |
| 8 | 18 | 1 | 2 | 43 | 31 | 1 | 1 | 162 | 0 | 0.5 | 2 | 0 | 3 | 1 |
| 9 | 23 | 1 | 2 | 34 | 9 | 0 | 1 | 174 | 0 | 1.6 | 2 | 0 | 2 | 1 |
| 10 | 20 | 1 | 0 | 28 | 69 | 0 | 1 | 160 | 0 | 1.2 | 2 | 0 | 2 | 1 |

## Code:

```
# Setting the dimensions of the plot
fig, ax = plt.subplots(figsize=(6, 6))
sns.countplot(x=df['age'], data=df)
plt.title("Category wise count of 'age'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

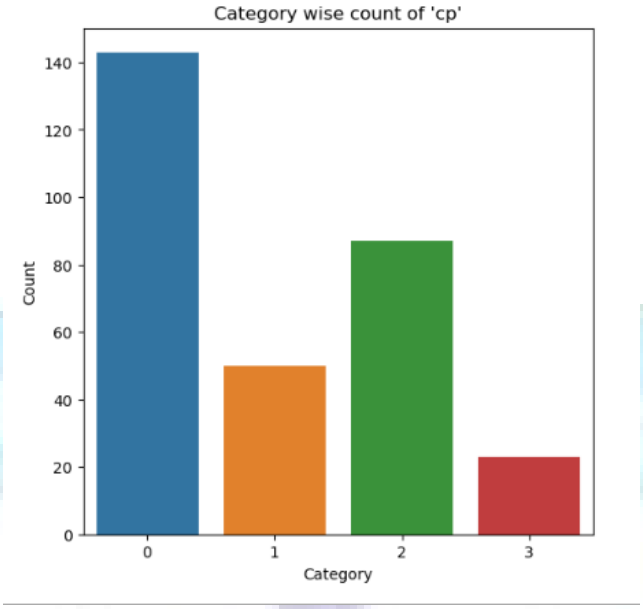
## Output:




## Code:

```
# Setting the dimensions of the plot
fig, ax = plt.subplots(figsize=(6, 6))
sns.countplot(x=df['age'], data=df)
plt.title("Category wise count of 'age'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

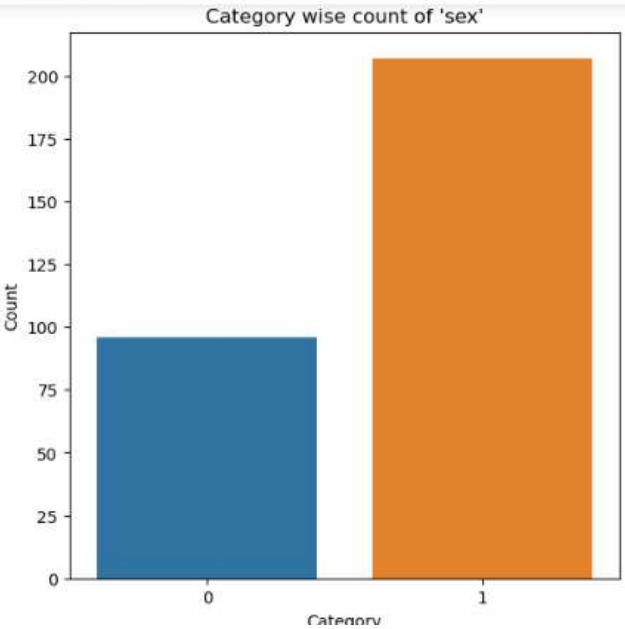
## Output:




## Code:

```
# Setting the dimensions of the plot
fig, ax = plt.subplots(figsize=(6, 6))
sns.countplot(x=df['sex'], data=df)
plt.title("Category wise count of 'sex'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

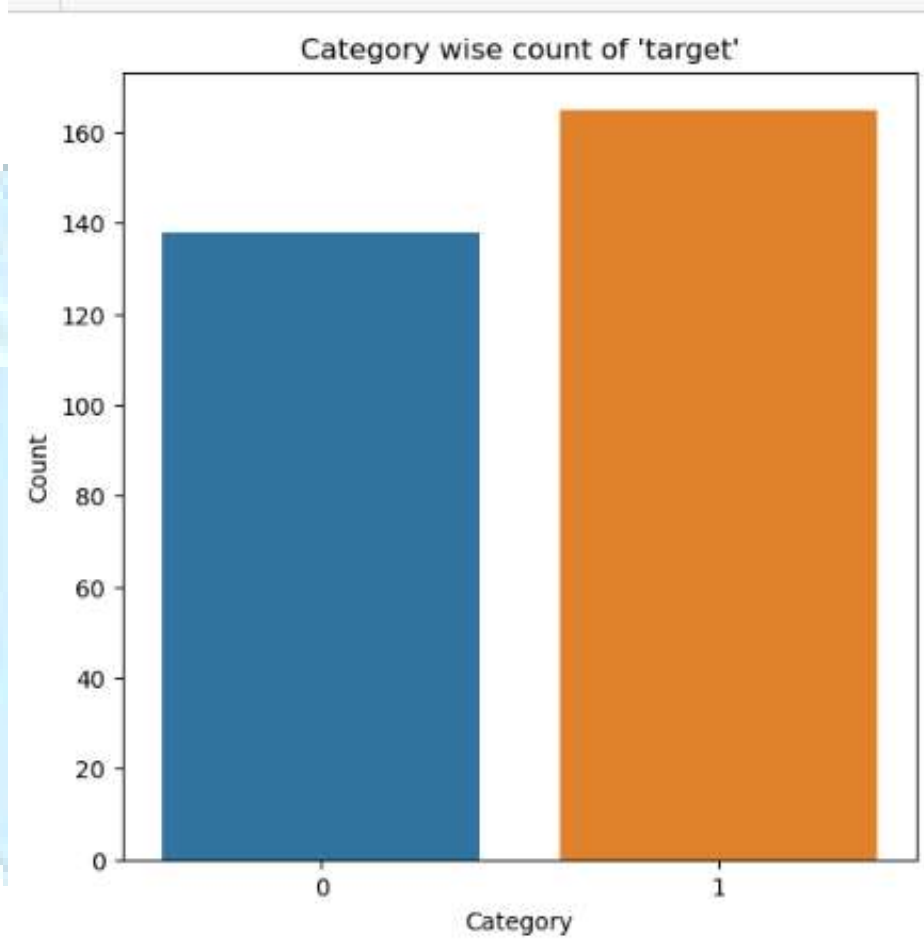
## Output:



## Code:

```
# Setting the dimensions of the plot
fig, ax = plt.subplots(figsize=(6, 6))
sns.countplot(x=df['target'], data=df)
plt.title("Category wise count of 'target'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

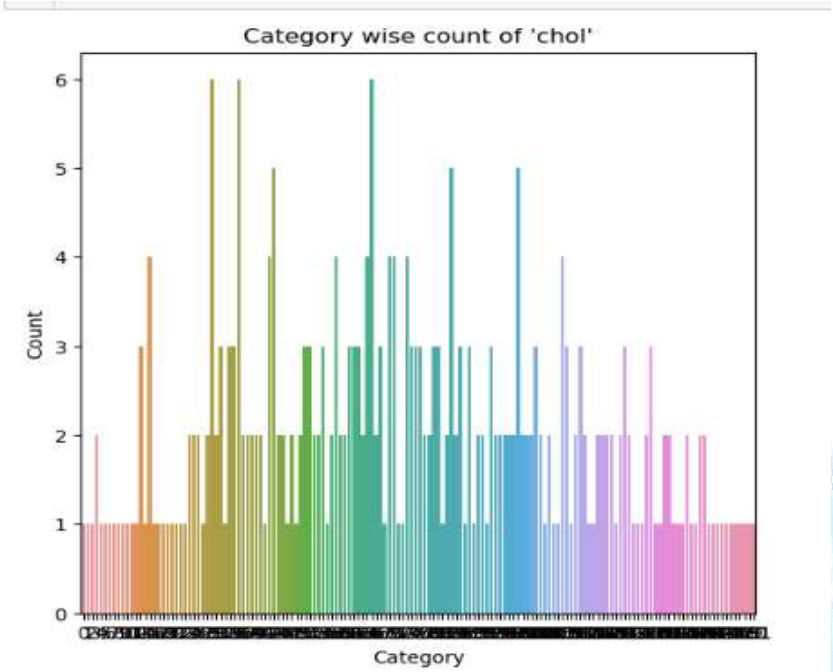
## Output:




## Code:

```
# Setting the dimensions of the plot
fig, ax = plt.subplots(figsize=(6, 6))
sns.countplot(x=df['chol'], data=df)
plt.title("Category wise count of 'chol'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

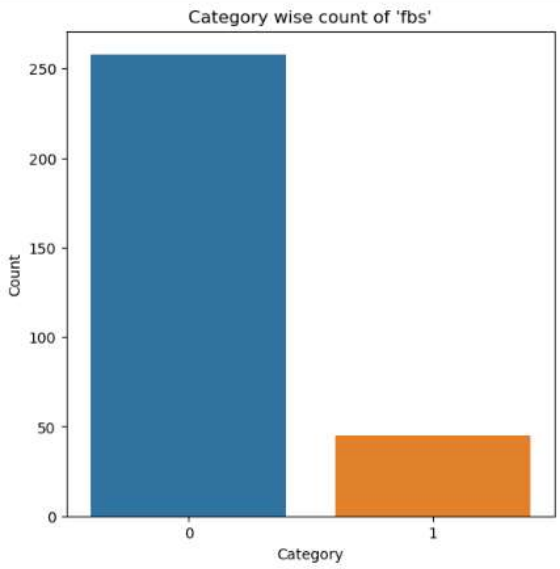
## Output:




## Code:

```
# Setting the dimensions of the plot
fig, ax = plt.subplots(figsize=(6, 6))
sns.countplot(x=df['fbs'], data=df)
plt.title("Category wise count of 'fbs'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

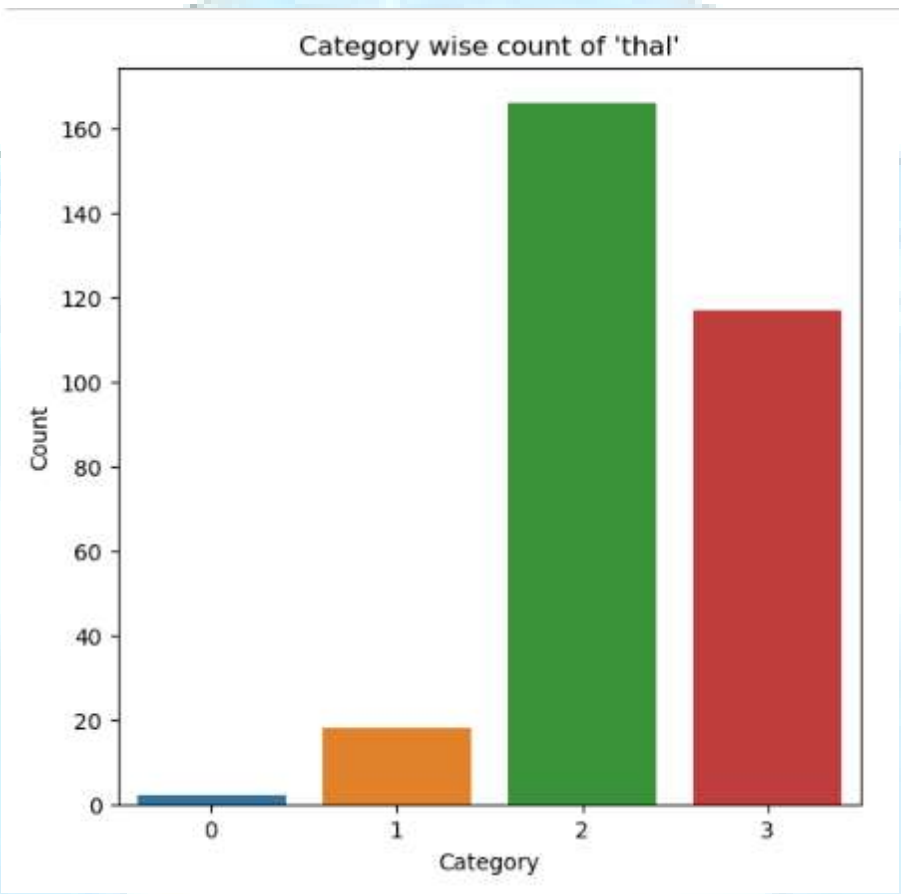
## Output:




## Code:

```
# Setting the dimensions of the plot
fig, ax = plt.subplots(figsize=(6, 6))
sns.countplot(x=df['thal'], data=df)
plt.title("Category wise count of 'thal'")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

**Output:**




**Code:**

```
X = df.drop('sex', axis=1)
y = df['sex']
classifier = MultinomialNB()
classifier.fit(X, y)
```

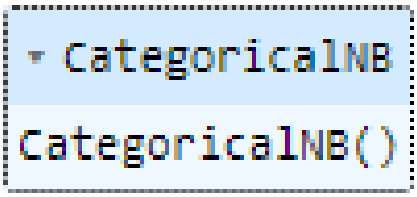
**Output:**

```
▾ MultinomialNB
MultinomialNB()
```

**Code:**

```
# Using CategoricalNB classifier
classifier = CategoricalNB()
classifier.fit(X, y)
```

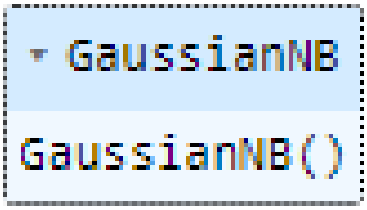
**Output:**




**Code:**

```
# Training algorithm with GaussianNB classifier
classifier_gaussian = GaussianNB()
classifier_gaussian.fit(X, y)
```

**Output:**




**Code:**

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix,
precision_score, recall_score, f1_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Initializing the MultinomialNB classifier
classifier = MultinomialNB()

# Training the classifier
classifier.fit(X_train, y_train)

# Predicting on the test set
y_pred = classifier.predict(X_test)

# Evaluating the model
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 score:", f1_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

**Output:**

```
Confusion Matrix:
 [[11 11]
 [12 27]]
Accuracy: 0.6229508196721312
Precision: 0.7105263157894737
Recall: 0.6923076923076923
F1 score: 0.7012987012987013
Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.50      0.49        22
           1       0.71      0.69      0.70        39

    accuracy                           0.62        61
   macro avg       0.59      0.60      0.60        61
weighted avg       0.63      0.62      0.62        61
```

# Practical No. 8

## Aim: Implement the K-NN Algorithm for classification or regression.

- **Apply K-NN Algorithm on the given dataset & predict the class or value for test data.**

## Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.style.use('ggplot')

df=pd.read_csv('D:\\ABC\\diabetes.csv')
df.head()
```

**Output: -**

Out[2]:

| | Pregnancies | Glucose | BloodPressure | SkinThickness | Insulin | BMI | DiabetesPedigreeFunction | Age | Outcome |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 6 | 148 | 72 | 35 | 0 | 33.6 | 0.627 | 50 | 1 |
| 1 | 1 | 85 | 66 | 29 | 0 | 26.6 | 0.351 | 31 | 0 |
| 2 | 8 | 183 | 64 | 0 | 0 | 23.3 | 0.672 | 32 | 1 |
| 3 | 1 | 89 | 66 | 23 | 94 | 28.1 | 0.167 | 21 | 0 |
| 4 | 0 | 137 | 40 | 35 | 168 | 43.1 | 2.288 | 33 | 1 |

df.shape

**Output: -**

```
In [3]: df.shape
Out[3]: (768, 9)
```

df.dtypes

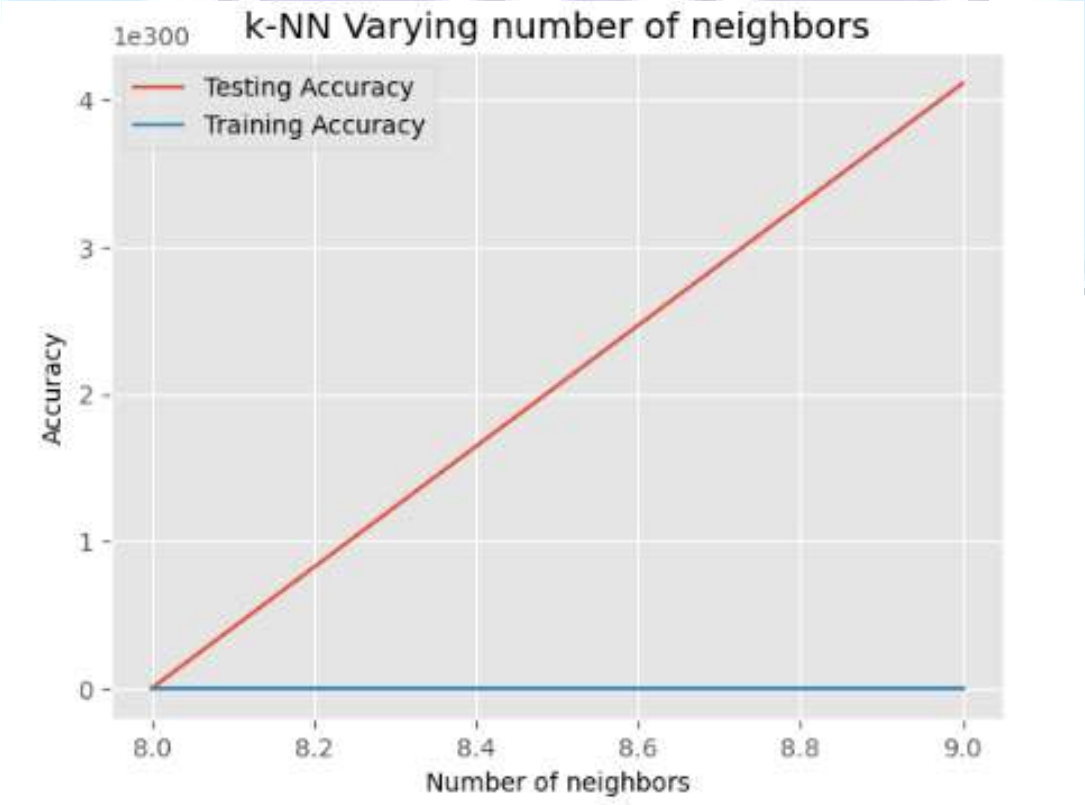
**Output: -**

```
In [4]: df.dtypes
Out[4]: Pregnancies                 int64
        Glucose                     int64
        BloodPressure               int64
        SkinThickness               int64
        Insulin                     int64
        BMI                       float64
        DiabetesPedigreeFunction  float64
        Age                         int64
        Outcome                     int64
        dtype: object
```

```
x=df.drop('Outcome',axis=1).values
y=df['Outcome'].values

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.4,random_state=42)
from sklearn.neighbors import KNeighborsClassifier
neighbors = np.arange(8,10)
train_accuracy = np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))
for i,k in enumerate(neighbors):
    #setup a knn classifier with a neighbors
    knn = KNeighborsClassifier(n_neighbors=k)
    #Fit the model
    knn.fit(x_train,y_train)
    #Compute accuracy on the training set
    train_accuracy[i]=knn.score(x_train,y_train)
    test_accuracy[i]=knn.score(x_test,y_test)
    plt.title('k-NN Varying number of neighbors')
    plt.plot(neighbors, test_accuracy, label='Testing Accuracy')
    plt.plot(neighbors, train_accuracy, label='Training Accuracy')
    plt.legend()
    plt.xlabel('Number of neighbors')
    plt.ylabel('Accuracy')
    plt.show()
```

**Output: -**



```
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(x_train,y_train)
KNeighborsClassifier(n_neighbors=7)
knn.score(x_test,y_test)
```

**Output: -**

```
In [10]: knn = KNeighborsClassifier(n_neighbors=7)
         knn.fit(x_train,y_train)
         KNeighborsClassifier(n_neighbors=7)
         knn.score(x_test,y_test)

Out[10]: 0.7142857142857143
```

## Code:

```
from sklearn.metrics import confusion_matrix
y_pred = knn.predict(x_test)
confusion_matrix(y_test,y_pred)
```

**Output: -**

```
In [11]: from sklearn.metrics import confusion_matrix
         y_pred = knn.predict(x_test)
         confusion_matrix(y_test,y_pred)

Out[11]: array([[163,  43],
                [ 45,  57]], dtype=int64)
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test,y_pred))
```

**Output: -**

```
In [12]: from sklearn.metrics import classification_report
         print(classification_report(y_test,y_pred))

                      precision    recall  f1-score   support

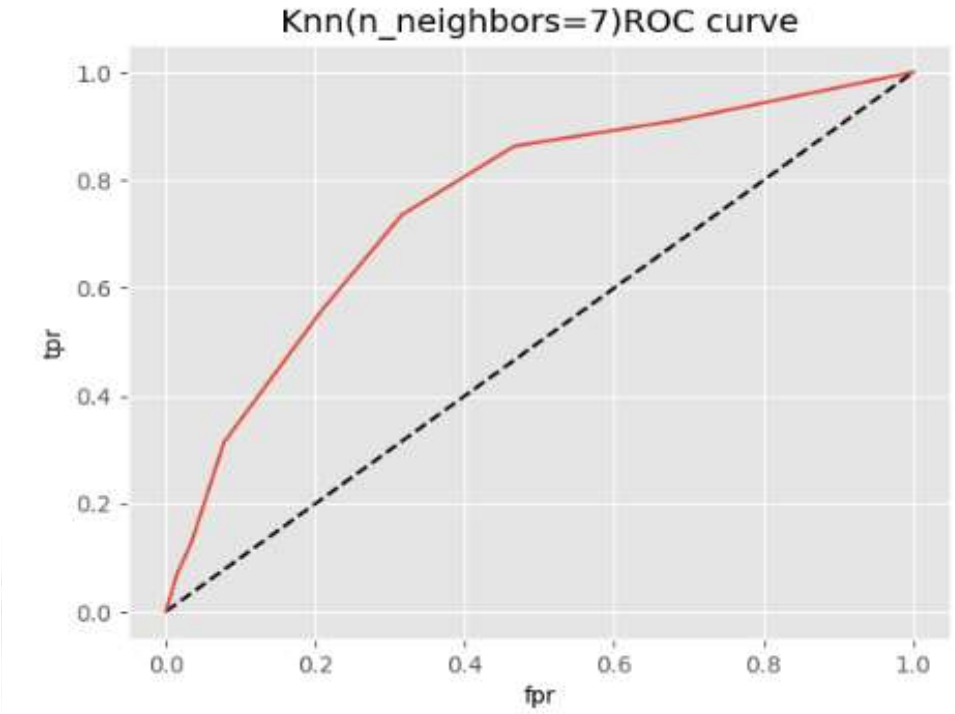
                   0       0.78      0.79      0.79       206
                   1       0.57      0.56      0.56       102

            accuracy                           0.71       308
           macro avg       0.68      0.68      0.68       308
        weighted avg       0.71      0.71      0.71       308
```

```
y_pred_proba = knn.predict_proba(x_test)[:,1]
from sklearn.metrics import roc_curve
fpr,tpr,thresholds=roc_curve(y_test,y_pred_proba)
plt.plot([0,1],[0,1],'k--')
```

```
plt.plot(fpr,tpr,label='Knn')
plt.xlabel('fpr')
plt.ylabel('tpr')
plt.title('Knn(n_neighbors=7)ROC curve')
plt.show()
```

**Output: -**




```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,y_pred_proba)
```

**Output: -**

```
In [14]: from sklearn.metrics import roc_auc_score
         roc_auc_score(y_test,y_pred_proba)

Out[14]: 0.7536645726251665
```

```
from sklearn.model_selection import GridSearchCV
param_grid = {'n_neighbors':np.arange(1,50)}
knn = KNeighborsClassifier()
knn_cv = GridSearchCV(knn,param_grid,cv=5)
knn_cv.fit(x,y)
```

**Output: -**

```
In [20]: knn = KNeighborsClassifier()
         knn_cv = GridSearchCV(knn,param_grid,cv=5)
         knn_cv.fit(x,y)

Out[20]: GridSearchCV
GridSearchCV(cv=5, estimator=KNeighborsClassifier(),
             param_grid={'n_neighbors': array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34,
       35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49])})
```

knn_cv.best_score_

**Output: -**

```
In [21]: knn_cv.best_score_
Out[21]: 0.7578558696205755
```

knn_cv.best_params_

**Output: -**

```
In [22]: knn_cv.best_params_
Out[22]: {'n_neighbors': 14}
```

# Practical No. 9

## Aim: Association Rule Mining

- **Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent itemsets.**
- **Generate association rules from the frequent itemsets and calculate their support and confidence.**
- **Interpret and analyze the discovered association rules.**

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 5GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

```
/kaggle/input/groceries-dataset/Groceries_dataset.csv
```

### Importing Libraries

### Program1:

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
try:
    import apyori
except ImportError:
    !pip install apyori
```

### Output:

```
Collecting apyori
  Using cached apyori-1.1.2.tar.gz (8.6 kB)
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Getting requirements to build wheel: started
  Getting requirements to build wheel: finished with status 'done'
  Preparing metadata (pyproject.toml): started
  Preparing metadata (pyproject.toml): finished with status 'done'
Building wheels for collected packages: apyori
  Building wheel for apyori (pyproject.toml): started
  Building wheel for apyori (pyproject.toml): finished with status 'done'
  Created wheel for apyori: filename=apyori-1.1.2-py3-none-any.whl size=5974 sha256=ae1dcce7b0f95212d9b9688b06e8e0b5d4c723e44e0fa6278017a2a950cb5e82
  Stored in directory: c:\users\admin\appdata\local\pip\cache\wheels\77\3d\a6\d317a6fb32be58a602b1e8c6b5d6f31f79322da554cad2a5ea
```

**Program 2:**

```
from apyori import apriori
import pandas as pd
df = pd.read_csv('Downloads/Groceries_dataset.csv', parse_dates=['Date'])
df.head()
```

**Output:**

| | Member_number | Date | itemDescription |
|---|---|---|---|
| 0 | 1808 | 2015-07-21 | tropical fruit |
| 1 | 2552 | 2015-01-05 | whole milk |
| 2 | 2300 | 2015-09-19 | pip fruit |
| 3 | 1187 | 2015-12-12 | other vegetables |
| 4 | 3037 | 2015-02-01 | whole milk |

**Program 3:**

```
df.isnull().any()
```

**Output:**

```
Member_number      False
Date               False
itemDescription    False
dtype: bool
```

**Program 4:**

```
all_products = df['itemDescription'].unique()
print("Total products: {}".format(len(all_products)))
```

**Output:**

```
Total products: 167
```

**Program 5:**

```
def distribution_plot(x, y, name=None, xaxis=None, yaxis=None):
    fig = go.Figure([
        go.Bar(x=x, y=y)
    ])
    fig.update_layout(
```

```
    title_text=name,
    xaxis_title=xaxis,
    yaxis_title=yaxis
  )
  fig.show()

x = df['itemDescription'].value_counts()
x = x.sort_values(ascending=False)
x = x[:10]

distribution_plot(x=x.index, y=x.values, yaxis="Count", xaxis="Products")
```

## Output:




## Program 6:

```
one_hot = pd.get_dummies(df['itemDescription'])
df.drop('itemDescription', inplace=True, axis=1)
df = df.join(one_hot)
```

## Output:

| | Member_number | Date | Instant food products | UHT-milk | abrasive cleaner | artif. sweetener | baby cosmetics | bags | baking powder | t |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1808 | 2015-07-21 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| 1 | 2552 | 2015-05-01 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| 2 | 2300 | 2015-09-19 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| 3 | 1187 | 2015-12-12 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| 4 | 3037 | 2015-01-02 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |

5 rows × 169 columns

## Transactions:-

### Program 7:

```
records = df.groupby(["Member_number", "Date"])[all_products[:]].apply(sum)
records = records.reset_index()[all_products]

def get_Pnames(x):
    for product in all_products:
        if x[product]> 0:
            x[product] = product
    return x

records = records.apply(get_Pnames, axis=1)
records.head()
```

### Output:

| | tropical fruit | whole milk | pip fruit | other vegetables | rolls/buns | pot plants | citrus fruit | beef | frankfurter | chicken | ... |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | whole milk | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... |
| 1 | 0 | whole milk | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... |
| 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... |
| 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | ... |

5 rows × 167 columns

### Program 8:

```
print("Total transactions: {}".format(len(records)))
```

### Output:

```
Total transactions: 14963
```

### Program 9:

```
x = records.values
x = [sub[~(sub == 0)].tolist() for sub in x if sub[sub != 0].tolist()]
transactions = x
```

transactions[0:10]

**Output:**

```
[['whole milk', 'pastry', 'salty snack'],
 ['whole milk', 'yogurt', 'sausage', 'semi-finished bread'],
 ['soda', 'pickled vegetables'],
 ['canned beer', 'misc. beverages'],
 ['sausage', 'hygiene articles'],
 ['whole milk', 'rolls/buns', 'sausage'],
 ['whole milk', 'soda'],
 ['frankfurter', 'soda', 'whipped/sour cream'],
 ['beef', 'white bread'],
 ['frankfurter', 'curd']]
```

## Association Rules

## Program 10:

```
rules = apriori(transactions, min_support=0.00030, min_confidence=0.05, min_lift=3,
min_length=2)
association_results = list(rules)

for item in association_results:
    pair = item[0]
    items = [x for x in pair]
    print("Rule: " + items[0] + " -> " + items[1])
    print("Support: " + str(item[1]))
    print("Confidence: " + str(item[2][0][2]))
    print("Lift: " + str(item[2][0][3]))
    print("=====================================")
```

**Output:**

Rule: fruit/vegetable juice -> liver loaf
Support: 0.00040098910646260775
Confidence: 0.12
Lift: 3.5276227897838903
==========

Rule: ham -> pickled vegetables
Support: 0.0005346521419501437
Confidence: 0.05970149253731344
Lift: 3.4895055970149254
==========

Rule: roll products -> meat
Support: 0.0003341575887188398
Confidence: 0.06097560975609757
Lift: 3.620547812620984
==========

Rule: misc. beverages -> salt
Support: 0.0003341575887188398
Confidence: 0.05617977528089888
Lift: 3.5619405827461437
==========

Rule: misc. beverages -> spread cheese
Support: 0.0003341575887188398
Confidence: 0.05
Lift: 3.170127118644068
==========

Rule: seasonal products -> soups
Support: 0.0003341575887188398
Confidence: 0.10416666666666667
Lift: 14.704205974842768
==========

Rule: spread cheese -> sugar
Support: 0.00040098910646260775
Confidence: 0.06
Lift: 3.3878490566037733
==========

Rule: bottled beer -> sausage
Support: 0.0003341575887188398
Confidence: 0.2
Lift: 3.3140642303433
==========

Rule: bottled beer -> hard cheese
Support: 0.0003341575887188398
Confidence: 0.17857142857142858
Lift: 3.9409502739148756
==========

Rule: canned beer -> brown bread
Support: 0.0003341575887188398

Confidence: 0.1388888888888889
Lift: 4.959891275523733
==========
Rule: canned beer -> brown bread
Support: 0.00040098910646260775
Confidence: 0.2222222222222222
Lift: 4.736625514403292
==========
Rule: frankfurter -> butter
Support: 0.0003341575887188398
Confidence: 0.10869565217391304
Lift: 3.0861727580232654
==========
Rule: canned beer -> butter milk
Support: 0.0003341575887188398
Confidence: 0.4166666666666667
Lift: 4.851815823605707
==========
Rule: canned beer -> frozen vegetables
Support: 0.0003341575887188398
Confidence: 0.25
Lift: 4.142580287929125
==========
Rule: canned beer -> whole milk
Support: 0.00040098910646260775
Confidence: 0.5454545454545454
Lift: 3.4539299042049785
==========
Rule: yogurt -> chewing gum
Support: 0.00040098910646260775
Confidence: 0.39999999999999997
Lift: 4.657743190661479
==========
Rule: pork -> yogurt
Support: 0.00040098910646260775
Confidence: 0.3
Lift: 3.4933073929961087

==========

Rule: frankfurter -> coffee
Support: 0.0003341575887188398
Confidence: 0.1388888888888889
Lift: 3.6782202556538843

==========

Rule: frankfurter -> coffee
Support: 0.0003341575887188398
Confidence: 0.13513513513513514
Lift: 3.578808897392969

==========

Rule: coffee -> sausage
Support: 0.0003341575887188398
Confidence: 0.2380952380952381
Lift: 3.9453145599325

==========

Rule: curd -> fruit/vegetable juice
Support: 0.0003341575887188398
Confidence: 0.5
Lift: 8.28516057585825

==========

Rule: margarine -> curd
Support: 0.0003341575887188398
Confidence: 0.3125
Lift: 5.178225359911406

==========

Rule: curd -> sausage
Support: 0.00046782062420637575
Confidence: 0.2058823529411765
Lift: 3.4115367077063388

==========

Rule: sausage -> hard cheese
Support: 0.0003341575887188398
Confidence: 0.2
Lift: 3.3140642303433

==========

Rule: rolls/buns -> pip fruit

Support: 0.0003341575887188398
Confidence: 0.5
Lift: 4.545261239368165
==========

Rule: shopping bags -> margarine
Support: 0.0003341575887188398
Confidence: 0.1724137931034483
Lift: 3.623353351414181
==========

Rule: margarine -> sausage
Support: 0.00040098910646260775
Confidence: 0.18749999999999997
Lift: 3.1069352159468435
==========

Rule: newspapers -> sausage
Support: 0.0003341575887188398
Confidence: 0.1851851851851852
Lift: 3.580007656235047
==========

Rule: onions -> tropical fruit
Support: 0.0003341575887188398
Confidence: 0.2777777777777778
Lift: 3.2345438824038046
==========

Rule: sausage -> waffles
Support: 0.0003341575887188398
Confidence: 0.05555555555555556
Lift: 3.001002807862014
==========

Rule: soft cheese -> other vegetables
Support: 0.0003341575887188398
Confidence: 0.2777777777777778
Lift: 3.2345438824038046
==========

Rule: pork -> sausage
Support: 0.00040098910646260775
Confidence: 0.2608695652173913

Lift: 3.037658602605312

==========

Rule: pastry -> whole milk
Support: 0.0003341575887188398
Confidence: 0.09090909090909091
Lift: 7.817659352142111

==========

Rule: whole milk -> rolls/buns
Support: 0.0003341575887188398
Confidence: 0.0625
Lift: 5.599925149700599

==========